

基于交替发射模式的CUDA加速SAR回波仿真软件

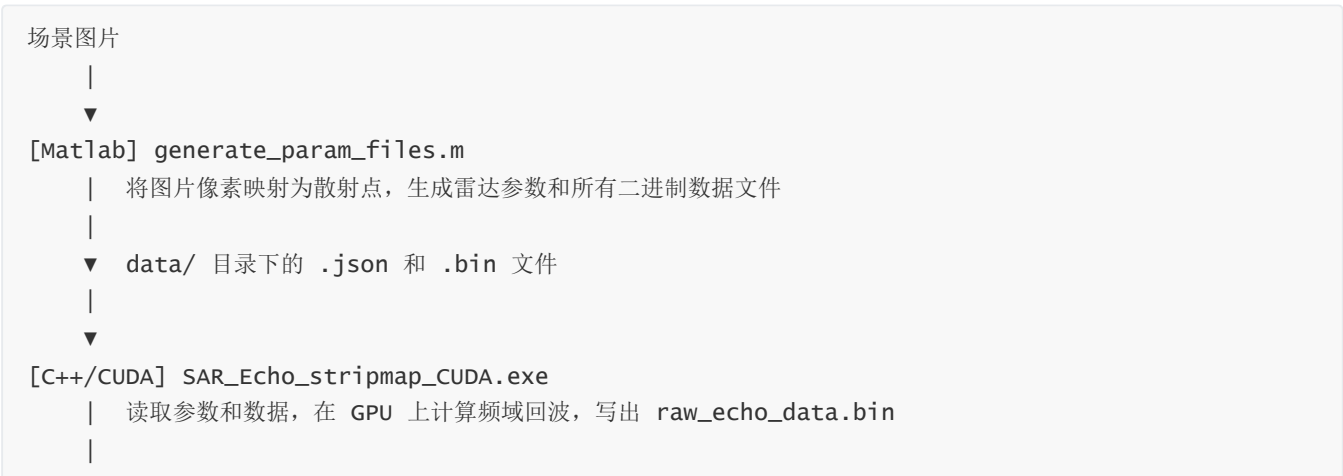
一、目录结构

SAR_Echo_stripmap_CUDA_release/	
├─ scrpits/	# Matlab 脚本（参数生成 + 成像）
│ └─ matlabFuncLib_v1.2/	# 依赖的 Matlab 函数库（需导入路径）
│ └─ generate_param_files.m	# 面目标仿真：生成所有输入数据文件
│ └─ generate_param_point_target.m	# 点目标仿真：用于验证系统响应
│ └─ read_rd_std_stripmap.m	# 回波读取 + RD 算法成像
│ └─ RDA_imaging.m	# RD 算法核心函数
│ └─ wf_s4_N2250.mat	# 预存的相位编码波形数据（4 路）
│ └─ *.png	# 示例场景图片
├─ SAR_Echo_stripmap_CUDA/	# C++/CUDA 回波仿真工程（主工程）
│ └─ echo_engine.cu	# CUDA 核心计算（GPU 加速的回波累加）
│ └─ echo_app.cpp	# 应用入口：文件读取、流程调度
│ └─ config.cpp / config.hpp	# JSON 参数解析
│ └─ readers.cpp / readers.hpp	# 二进制数据文件读取
│ └─ types.hpp	# 数据类型定义
│ └─ SAR_Echo_stripmap_CUDA.sln	# Visual Studio 解决方案文件
├─ base/	# 与主工程代码相同，备用工程
└─ data/	# 运行时数据目录（由 Matlab 脚本生成）
└─ radar_params.json	# 雷达系统参数
└─ running_params.json	# 运行配置（GPU 编号、批量大小等）
└─ target_list.bin	# 目标散射点列表（由 Matlab 生成）
└─ platform_pos.bin	# 平台轨迹（由 Matlab 生成）
└─ waveforms_f0.bin	# 频域波形矩阵（由 Matlab 生成）
└─ waveform_codebook.bin	# 发射波形编码表（由 Matlab 生成）
└─ raw_echo_data.bin	# 仿真输出的原始回波（由 C++ 生成）

二、软件设计原理

1. 总体仿真流程

本软件对条带式 SAR（Stripmap SAR）的原始回波进行**时域仿真**，即直接从物理散射模型出发，逐脉冲计算每个散射点对雷达回波的贡献。整体流程如下：



```
▼ raw_echo_data.bin (每个方位脉冲的频域回波, 已 fftshift)
|
▼
[Matlab] read_rd_std_striple.m
| 距离压缩 + RD 算法方位压缩, 成像并保存结果
▼
output.png (SAR 图像)
```

2. 回波仿真的物理原理

2.1 频域回波计算

仿真在距离频域中进行（即对每个方位脉冲，计算其频域回波 $S(f_r)$ ），而非在时域逐采样点计算。对于第 a 个方位脉冲，其频域回波为所有散射点贡献的叠加：

$$S(f_r, t_a) = \sum_k \sigma_k \cdot W(f_r) \cdot \exp(-j2\pi [f_r \cdot \Delta\tau_k + f_c \cdot \tau_k])$$

其中：

- $\sigma_k = \sigma_{k,r} + j\sigma_{k,i}$ ：第 k 个散射点的复散射系数
- $W(f_r)$ ：发射波形的频域表达（LFM 或其他调制波形）
- f_c ：载频， f_r ：距离频率偏移量
- $\tau_k = (R_{tx,k} + R_{rx,k})/c$ ：发射→目标→接收的总传播时延
- $\Delta\tau_k = \tau_k - 2R_0/c$ ：相对参考斜距 R_0 的时延差

为什么在频域计算？ 频域回波可以直接用于后续的距离压缩（频域匹配滤波），省去了时域回波到频域的额外 FFT 步骤，同时对内存带宽更友好。

2.2 方位向波门 (az_gate)

实际上，对于给定方位位置的雷达，只有处于天线波束照射范围内的散射点才对当前脉冲有贡献。代码中通过 `az_gate_half`（波束半宽对应的方位距离）进行筛选：

```
如果 |x_radar - x_target| >= az_gate_half, 则跳过该散射点
```

这一判断大幅减少了无效计算，是仿真效率的关键优化之一。

2.3 混合精度计算

相位计算（尤其是远距离目标）对浮点精度非常敏感。代码提供两种精度模式，通过 `running_params.json` 中的 `enable_mixed_precision` 控制：

- **全 double 模式** (`false`)：波形数据与几何/相位计算均使用 double 精度，精度最高
- **混合精度模式** (`true`)：波形数据用 float 存储（节省显存带宽），几何距离和相位累加仍用 double，在精度与速度之间取得平衡

3. CUDA 并行加速原理

SAR 回波仿真的计算量非常大：以本软件默认参数为例，需要对 4000 个方位脉冲 × 4000 个距离频点 × 数万个散射点进行计算，总浮点运算量高达数十亿次。

GPU（CUDA）加速的核心思路是：**每个（距离频点, 方位脉冲）组合都是相互独立的**，可以同时由成千上万个 GPU 线程并行处理。

代码中，一个 CUDA 线程负责计算一个输出点（距离频点 `rg`，方位脉冲 `az`）的回波值——遍历所有散射点，累加其贡献。GPU 上同时运行大量这样的线程，从而将原本需要串行执行的循环变为并行。

为避免一次性将所有方位脉冲数据搬运到显存造成内存溢出，代码采用**批处理（batch）**策略：每次只处理 `batch_cols`（默认 16~256）个方位脉冲，计算完成后将结果写回硬盘，再处理下一批。

4. 交替发射模式（多波形）

交替发射是指雷达在不同方位脉冲上轮流使用多种不同的发射波形（例如 4 路相位编码波形），这是现代多波束/多极化 SAR 的重要工作模式。

本软件通过**波形编码表（codebook）**实现这一机制：

- `waveforms_f0.bin`：存储所有发射波形的频域矩阵，共 N_{trans} 行，每行对应一种波形
- `waveform_codebook.bin`：长度为 N_{az} 的整数数组，`codebook[a]` 表示第 a 个方位脉冲使用第几号波形（0 起索引）
- 在 GPU 核函数中，每个线程首先查表 `wf_idx = codebook[az]`，再从波形矩阵中取出对应行参与计算

默认编码表：若 `waveform_codebook.bin` 不存在，程序自动使用轮换方式 $(0, 1, 2, \dots, N_{trans} - 1, 0, 1, \dots)$ 。

配置多波形：在 `generate_param_files.m` 中，修改 `waveforms_matrix`（每行一个波形，列数必须等于 `N_rg`）和 `codebook` 数组，即可灵活配置任意交替发射模式。示例中加载的 `wf_s4_N2250.mat` 即为一组 4 路相位编码波形（`commsignal`，4 行）。

5. 成像算法（RD 算法）

`read_rd_std_stripmap.m` 调用 `RDA_imaging.m`，实现标准的距离-多普勒（Range-Doppler）成像算法：

- 距离压缩：**在距离频域将回波与参考波形做共轭相乘（匹配滤波），变换回时域得到距离压缩结果
- 方位向 FFT：**将距离压缩后的数据变换至多普勒域
- 距离徙动校正（RCM）：**在多普勒域对每个多普勒频率计算其距离徙动量并进行相位补偿
- 方位压缩：**与方位参考函数做匹配滤波，变换回时域，完成成像

三、环境配置

开发环境要求

工具	推荐版本
Matlab	2021b ~ 2023b（或更新版本）

工具	推荐版本
Visual Studio	2022（MSVC 编译器，含 C++ 桌面开发工作负载）
CUDA Toolkit	11.4 / 12.6 / 13.0 均可
Nvidia 驱动	与所安装 CUDA 版本配套

注意：CUDA 版本与显卡架构无特殊限制。若需调整，可在 Visual Studio 中修改 `.vcxproj` 的 CUDA 目标架构（`compute_XX,sm_XX`）以与本机显卡对齐。

Matlab 环境配置

在使用任何脚本前，需将 `matlabFuncLib_v1.2` 目录添加到 Matlab 路径：

```
addpath('scrpits/matlabFuncLib_v1.2');
```

或在 Matlab 主界面中，通过「主页 → 设置路径 → 添加并包含子文件夹」选择该目录，并保存。

四、使用流程

步骤 1：配置仿真参数并生成数据文件

打开 `scrpits/generate_param_files.m`，根据需要修改以下参数：

雷达系统参数（文件开头 `cfg` 结构体）：

参数	含义	默认值
<code>cfg.fc</code>	载频 (Hz)	10 GHz
<code>cfg.B</code>	信号带宽 (Hz)	100 MHz
<code>cfg.fs</code>	采样率 (Hz)	150 MHz
<code>cfg.PRF</code>	脉冲重复频率 (Hz)	2000
<code>cfg.Tp</code>	脉冲宽度 (s)	15 μ s
<code>cfg.va</code>	平台速度 (m/s)	450
<code>cfg.H</code>	平台飞行高度 (m)	3000
<code>cfg.R0</code>	参考斜距 (m)	10000
<code>cfg.N_az</code>	方位脉冲数	4000

场景图片：修改 `img_path` 指向目标场景的灰度图（PNG 格式），图片的像素灰度值将被映射为散射系数幅度。

多波形配置：如需使用交替发射，修改 `waveforms_matrix`（每行一种波形的频域表示）和 `codebook` 数组。

运行脚本后，data/ 目录下会自动生成以下文件： radar_params.json、 running_params.json、 target_list.bin、 platform_pos.bin、 waveforms_f0.bin、 waveform_codebook.bin

步骤 2：配置 GPU 运行参数

打开 data/running_params.json，按需调整：

```
{
  "device_id": 0,
  "batch_cols": 256,
  "enable_mixed_precision": false,
  "enable_random_phase": true,
  "logger_level": "INFO"
}
```

参数	说明
device_id	GPU 编号，多卡机器可指定（从 0 开始）
batch_cols	每批处理的方位脉冲数，越大越快但占用更多显存，建议 64~512
enable_mixed_precision	混合精度开关，true 速度更快，false 精度更高
enable_random_phase	是否为散射系数叠加随机初相（使图像更接近真实 SAR 纹理）

步骤 3：编译并运行回波仿真

1. 用 Visual Studio 打开 SAR_Echo_stripmap_CUDA.sln
2. 选择配置（Debug 或 Release）→ 「重新生成解决方案」
3. 运行程序（F5 或直接运行 .exe）

程序会自动在 ./data、 ../data、 ../../data 路径下查找参数文件。运行过程中控制台将显示进度：

```
[INFO] 读取参数成功：N_rg=4000，N_az=4000
[INFO] 波形行数 N_trans=4
[INFO] 目标数量：35820
[INFO] progress: 256/4000 (6%)
...
[INFO] 处理完成，输出文件：data\raw_echo_data.bin
```

完成后，data/raw_echo_data.bin 即为仿真得到的原始频域回波数据。

也可通过命令行指定路径：

```
SAR_Echo_stripmap_CUDA.exe --data-dir D:\my_data --output D:\my_data\echo.bin
```

步骤 4：成像

回到 Matlab，运行 `scrpits/read_rd_std_stripmap.m`。脚本将自动读取 `data/` 目录下的回波和参数文件，依次完成距离压缩、方位压缩，并将最终 SAR 图像保存为 `output.png`。

五、数据文件格式说明

所有二进制文件均为小端（little-endian）double（float64）格式，按以下规则存储：

文件名	格式	说明
<code>target_list.bin</code>	<code>[x, y, z, cr, ci] × N_target</code> ，double	散射点坐标及复散射系数
<code>platform_pos.bin</code>	<code>[tx_x, tx_y, tx_z, rx_x, rx_y, rx_z] × N_az</code> ，double	各脉冲的发射/接收天线位置
<code>waveforms_f0.bin</code>	<code>[Re, Im] × N_rg × N_trans</code> ，行块交错，double	频域波形矩阵，每行一种波形
<code>waveform_codebook.bin</code>	<code>int32 × N_az</code>	编码表，值为 0 起索引的波形编号
<code>raw_echo_data.bin</code>	<code>[Re, Im] × N_rg × N_az</code> ，double	频域回波（列序，已 fftshift）

六、常见问题

Q：运行 C++ 程序时提示找不到 `radar_params.json`？ A：程序默认依次搜索 `data`、`../data`、`../../data` 等相对路径。请确认已运行 Matlab 脚本生成数据文件，或使用 `--data-dir` 参数显式指定路径。

Q：Matlab 提示 `myfft` 未定义？ A：需要先将 `scrpits/matlabFuncLib_v1.2/` 添加到 Matlab 的搜索路径（`addpath`）。

Q：`waveforms_matrix` 列数与 `N_rg` 不匹配的错误？ A：波形矩阵的列数必须严格等于 `cfg.N_rg`，即距离采样点数。若更换波形，需确保长度对齐，不足部分可用零补齐（脚本中已有示例：`[zeros(4,875) commSignal zeros(4,875)]`）。

Q：CUDA 运行时报错 `cudaSetDevice` 失败？ A：确认机器安装了 NVIDIA 驱动和 CUDA Toolkit，并检查 `running_params.json` 中 `device_id` 是否超出实际 GPU 数量。

Q：如何仅验证点目标响应（不使用图片）？ A：运行 `generate_param_point_target.m` 替代 `generate_param_files.m`，该脚本在场景中心放置单个点目标，适合用于测量系统的 ISLR、PSLR 等指标。成像后在 `read_rd_std_stripmap.m` 中将 `is_point_eg` 设为 `true` 可进行点目标剖面分析。